

A Self-Evolving Reinforcement Learning System for Agentic Large Language Models

Hao Sun

School of Smart Governance, Renmin University of China

摘要

Abstract Large language model (LLM) agents accomplish tasks through multi-step tool use in real environments such as code sandboxes and file systems, where reinforcement learning (RL) has become the dominant approach for continuous improvement. However, RL for interactive agents faces a fundamental bottleneck: both training data and trustworthy rewards are difficult to obtain at scale through human effort. Real user data accumulates slowly with narrow coverage, and rewards that rely solely on agent self-reports invite reward hacking, where the model learns to falsely claim task completion. We present a *self-evolving* RL system for agentic LLMs that enables agents to autonomously generate training data, obtain trustworthy rewards, and produce next-step tasks without human annotation. The system couples (i) cross-step self-evolving task generation via three collaborating agents and true sandbox-state inheritance; (ii) a diff-driven reward that anchors scoring on deterministic environment state changes rather than self-reports; (iii) an oversample–eliminate–select mechanism that keeps only the gradient-optimal trajectory subset per step; and (iv) failure-case-driven self-evolution of the evaluator and infrastructure. Validated end-to-end on Qwen3.5-9B, the full pipeline runs from a small seed set with no external data beyond the initial seed, and offline evaluation confirms a genuine capability gain over the untrained base model.

Keywords LLM agents, reinforcement learning, self-evolution, reward hacking, GRPO

1 Introduction

As LLMs expand in capability and application, the conventional training paradigm that relies on hand-crafted tasks and fixed datasets faces a scaling bottleneck. Fixed data cannot keep pace with an ever-growing task space, and interactive-agent RL continuously consumes fresh trajectories, making data acquisition a key constraint on training scale and sustained iteration. *Self-evolution* has emerged as a response: a model, with minimal human intervention, keeps producing new experience and improving itself through a “generate–execute–evaluate–feedback–update” loop [1, 2].

Self-evolution unfolds on two levels. At the **model level**, a model generates its own data, scores itself, and fine-tunes on filtered experience—as in self-play, self-rewarding, RLAIIF, rejection sampling, and evolutionary search [3–6]. At the **agent level**,

an agent accumulates experience in an environment and improves its memory, skills, tool use, prompts, and planning, up to its underlying parameters. Our system spans both: the model autonomously generates data, scores itself against real environment state, and updates its parameters by RL, while the agent accumulates experience in a sandbox and drives the continual evolution of evaluator prompts and tooling from failure cases.

When task generation, reward evaluation, and policy update are all automated, the loop also incurs new risks: a biased evaluator or low-quality data can be amplified through subsequent rounds [7, 8]. An agent may earn high reward by matching the evaluator’s preferences without truly achieving the task. A self-evolving system therefore needs not only sustained data generation but also rewards anchored in environment execution results, plus quality control over candidate experience.

We build a self-evolving RL system for agentic LLMs. Starting from a small set of human seed tasks, the agent executes in a sandbox, obtains trajectories and environment state changes, and autonomously generates subsequent tasks; rewards are aided by environment state diffs; candidate trajectories are filtered by oversampling, quality elimination, and group selection before policy update; and the updated model drives the next round. Our contributions are:

- **Cross-step self-evolving task generation** that produces all post-seed tasks by growing each new task on the previous step’s real outputs.
- **Multi-agent autonomous data generation** that makes sampling, evidence collection, and follow-up fully self-driven.
- **Diff-driven reward** that anchors scoring on deterministic environment state diffs, suppressing reward hacking at the evidence level.
- **Failure-case-driven self-evolution** of the evaluator and infrastructure, extending self-evolution from tasks and policy to the evaluator and tooling.
- An **end-to-end validation** on Qwen3.5-9B showing the loop is self-sustaining and yields a real capability gain.

2 System Overview

The system comprises three collaborating agents (an *actor*, an *observer*, and a *questioner*), a code sandbox, an external frozen judge [15], and a GRPO trainer [9]. The actor solves tasks via ReAct-style reasoning and acting [10] with executable code actions [11]. All custom sampling and selection logic is attached through the official injection points of the RL framework, keeping the system decoupled from the framework core. Fig. 1 shows the overall architecture.

A self-evolving process is modeled as repeated application of a transition operator $\mathcal{T}$ on the system state $\sigma_t = (\theta_t, s_t, Q_t, \Phi_t)$ —policy parameters,

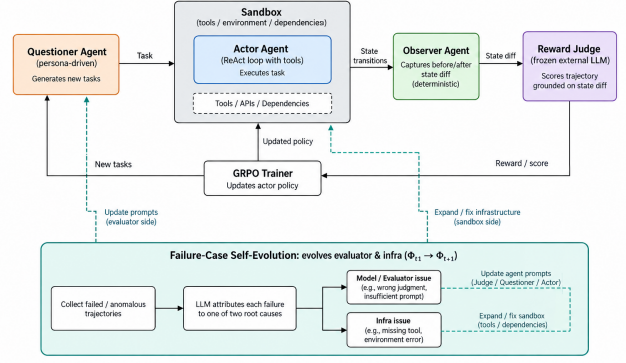


图 1: Overall system architecture: three collaborating agents, a code sandbox, a frozen judge, and a GRPO trainer connected through framework injection points.

sandbox state, task set, and evaluator/infrastructure state, respectively. Within one step, the main loop runs in three stages:

$$Q_t \xrightarrow{\pi_{\theta_t}} \tau_t \xrightarrow{\text{Observer}} d_t, \quad (1)$$

$$d_t \xrightarrow{\text{Judge}(\cdot; \Phi_t)} R_t \xrightarrow{\text{GRPO}} \theta_{t+1}, \quad (2)$$

$$(\tau_t, d_t) \xrightarrow{\text{Questioner}} Q_{t+1}. \quad (3)$$

Beyond the main loop, the evaluator/infrastructure state Φ_t evolves at a lower frequency, driven by an accumulated failure-case set $\mathcal{B}_t$. Traditional RL is the degenerate case where $Q_t \equiv Q_0$, $s_t \equiv s_0$, $\Phi_t \equiv \Phi_0$ and only θ_t evolves; our system evolves all four dimensions.

3 Method

3.1 Cross-Step Self-Evolving Task Generation

This mechanism lets the system produce every post-seed task without external supply. The core idea is to *grow the next task on the previous step’s real execution results*. After each step, for every surviving group the system takes the best trajectory and its sandbox state; an observer optionally summarizes the “intent–artifact–gap” of that trajectory; the sandbox file-system state is inherited into a fresh sandbox (true state inheritance, not a textual description); and a persona-driven questioner generates

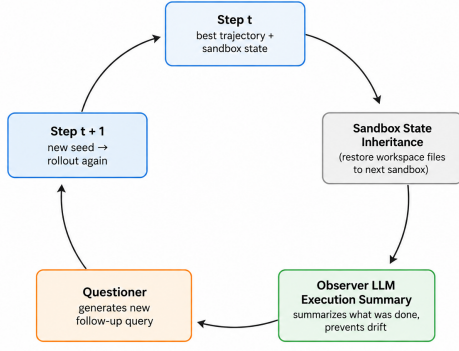


图 2: Cross-step self-evolution: the best trajectory’s sandbox state is inherited, and a questioner generates the next task from summarized experience.

the next seed. Fig. 2 illustrates the loop.

We make the execution summary *stochastically enabled*, yielding two complementary task modes: a *continuation mode* (summary on) that extends prior work into a long-horizon chain, and a *fresh-task mode* (summary off) that poses a new task on the inherited environment alone. Using a summary rather than the raw trajectory is deliberate: replaying the old policy’s verbatim text into the new policy’s context would inject off-policy bias and distribution shift; the summary keeps only task-level information stripped of the old policy’s generation traces.

3.2 Multi-Agent Autonomous Data Generation

A single sampling becomes a complete “ask–execute–attest” interaction driven by three agents with separated responsibilities (Fig. 3). The **actor** (the trained policy) solves tasks via ReAct in the sandbox. The **observer** takes read-only snapshots before and after execution and computes a deterministic state diff; it relies only on environment evidence, never on the actor’s self-report. The **questioner** plays a persona-driven simulated user and generates next-step seeds. Because task solving and follow-up are open-ended generation while attestation must be objective, this “capability vs. determinism” separation is what makes diff-driven reward trustworthy. From a data-source view, training data emerges from the mutual interplay among these units—a form of

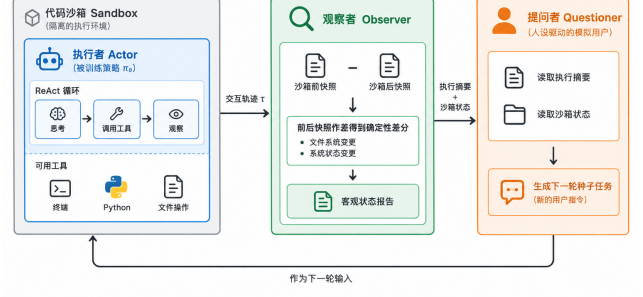


图 3: Multi-agent collaborative data generation: actor, observer, and questioner chained on one sandbox session.

self-play grounded in a real sandbox and adjudicated by environment state rather than free text. The use of role-specialized agents to generate interaction data follows the spirit of CAMEL [14] and multi-role self-evolution such as R-Zero [16].

3.3 Oversample–Eliminate–Select

To improve per-step signal quality, inference and training are decoupled. Each step oversamples S seeds $\times n$ rollouts; groups whose maximum reward is both in the bottom 20% and below a threshold are eliminated (at least 80% retained); the surviving R groups are ranked by mean absolute intra-group advantage, and the top N groups enter GRPO. This “dynamic sampling, filter invalid groups” idea is in line with DAPO [12]. The rationale: GRPO gradients are driven by intra-group advantage, so zero-variance (all-pass/all-fail) groups contribute no effective update. Selecting the highest-signal subset maximizes expected policy improvement under a fixed training budget.

3.4 Diff-Driven Reward

The reward’s evidence base is shifted from “what the agent says” to “what actually happened in the environment.” A frozen judge scores from the observer’s deterministic state diff (file-system and system-state changes); the actor’s self-report is used only for cross-checking. Scoring is multiplica-

tive over multiple dimensions (task completion, correctness, trajectory quality), so a zero on any key dimension yields zero overall; a safety dimension is a binary gate. If the diff is empty (no environment effect), the system short-circuits to zero without calling the judge. Formally, for any behavior change δ that alters only the self-report but not the state diff ($d(\delta) = d$), we have $\mathbb{E}[R \mid \delta] = \mathbb{E}[R]$: pure verbal hacking is fully blocked, and any reward increase requires a real change in the environment.

3.5 Failure-Case-Driven Self-Evolution of Evaluator and Infrastructure

Beyond tasks and policy, a complete self-evolving agent should evolve its *evaluator* (scoring/questioning prompts) and *infrastructure* (tools, sandbox, execution environment). Prior work shows failures can be attributed and repaired via verbal reflection [18] and that missing capabilities can be distilled into a reusable skill library [17]. Building on this, the system periodically collects failure cases (execution failures, reward anomalies, claim–diff contradictions). Every 10 steps or once 20 cases accumulate, an LLM attributes each case to the *model side* (policy planning, tool use, or plan selection) or the *infrastructure side* (missing dependency, missing tool, short timeout, harness defect). Model-side failures trigger incremental prompt revisions; infrastructure-side failures trigger tooling expansion or distillation of missing capabilities into reusable skills.

3.6 Self-Evolution Stability Control

Running unattended, the system monitors five collapse indicators on the existing compute path: reward std, advantage std, policy KL, training loss (NaN/Inf), and surviving-group count. Any single trigger terminates training, since each corresponds to a distinct failure mode under which further steps would only waste compute.

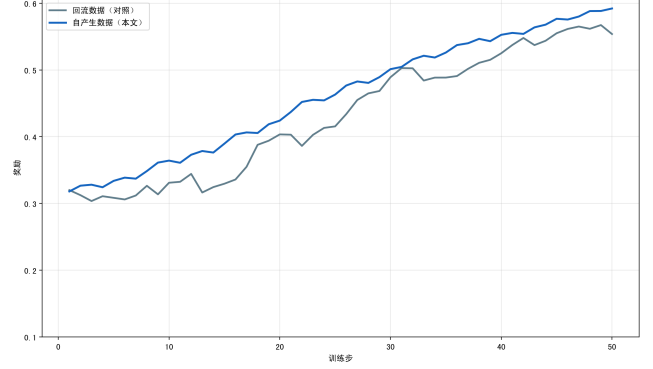


图 4: Reward-mean over training steps (EMA-smoothed): self-generated data (ours) vs. replay-data baseline.

4 Experiments

4.1 Setup

The base model is Qwen3.5-9B with standard GRPO; inference and training are asynchronously decoupled. Formal training uses 2 nodes $\times$ 8 H800 GPUs, $N=64$ groups per step, running 50 steps (manually stopped for comparability with conventional RL; the system itself imposes no step cap beyond the stability criterion). All results below come from *one* end-to-end run, observed along different dimensions.

4.2 Self-Sustaining Loop and Reward

The oversample amount S shrinks monotonically and stabilizes near N without collapsing to zero, and cross-step seed generation succeeds at every step—confirming a self-sustaining loop under no external data supply. Fig. 4 compares reward-mean curves of self-generated data against a replay-data baseline under an identical GRPO pipeline. Self-generated data rises steadily from about 0.33 to about 0.55, tracking and slightly exceeding the baseline: its training value is no weaker than real replay data.

A companion signal, the intra-group reward std, gradually narrows for self-generated data (a mild variance collapse / Echo-Trap [13] precursor) while the baseline stays flat—self-generated data trends toward homogeneity across rounds, whereas externally injected replay data sustains diversity.

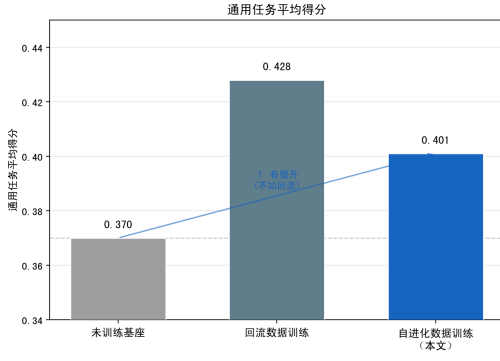


图 5: Average score on general tasks for the three models; the offline benchmark is independent of the training reward.

4.3 Capability Evaluation

Process metrics alone cannot prove a genuine capability gain: since the reward scale itself evolves, higher reward may merely reflect the system inflating its own score. We therefore use an offline benchmark (ClawEval text tasks) as a ruler *independent* of the training loop, comparing three models: the untrained base, a replay-data-trained model, and our self-evolution-trained model. As shown in Fig. 5, our model scores 0.401, clearly above the untrained base (0.370); because this evaluation is independent of the training reward, the gain reflects a real improvement in capability rather than inflated training metrics. Generation diversity drops slightly (0.285 vs. 0.294 for base), consistent with the variance collapse observed during training, so process and endpoint metrics agree.

5 Conclusion

We presented a self-evolving RL system for agentic LLMs that spans both the model level and the agent level, extending the object of self-evolution from “policy” alone to “task, policy, evaluator, and infrastructure.” The system autonomously generates data, anchors rewards on deterministic environment state diffs, filters trajectories for gradient quality, and evolves its evaluator and tooling from failure cases. End-to-end on Qwen3.5-9B, a small seed set drives complete RL training with no external data, and an independent benchmark confirms a real ca-

pability gain over the untrained base model. Multi-modal extension, diversity regularization, and larger-scale training are natural next directions.

参考文献

- [1] Z. Tao *et al.*, “A survey on self-evolution of large language models,” arXiv:2404.14387, 2024.
- [2] J. Huang *et al.*, “Large language models can self-improve,” arXiv:2210.11610, 2022.
- [3] Z. Chen *et al.*, “Self-play fine-tuning converts weak language models to strong language models,” arXiv:2401.01335, 2024.
- [4] W. Yuan *et al.*, “Self-rewarding language models,” arXiv:2401.10020, 2024.
- [5] H. Lee *et al.*, “RLAIF: Scaling reinforcement learning from human feedback with AI feedback,” arXiv:2309.00267, 2023.
- [6] A. Zhao *et al.*, “Absolute Zero: Reinforced self-play reasoning with zero data,” arXiv:2505.03335, 2025.
- [7] J. Skalse *et al.*, “Defining and characterizing reward hacking,” in *NeurIPS*, 2022.
- [8] L. Gao, J. Schulman, and J. Hilton, “Scaling laws for reward model overoptimization,” arXiv:2210.10760, 2022.
- [9] Z. Shao *et al.*, “DeepSeekMath: Pushing the limits of mathematical reasoning in open language models,” arXiv:2402.03300, 2024.
- [10] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” arXiv:2210.03629, 2022.
- [11] X. Wang *et al.*, “Executable code actions elicit better LLM agents,” arXiv:2402.01030, 2024.
- [12] Q. Yu *et al.*, “DAPO: An open-source LLM reinforcement learning system at scale,” arXiv:2503.14476, 2025.
- [13] Z. Wang *et al.*, “RAGEN: Understanding self-evolution in LLM agents via multi-turn reinforcement learning,” arXiv:2504.20073, 2025.

- [14] G. Li *et al.*, “CAMEL: Communicative agents for ‘mind’ exploration of large language model society,” arXiv:2303.17760, 2023.
- [15] L. Zheng *et al.*, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” arXiv:2306.05685, 2023.
- [16] C. Huang *et al.*, “R-Zero: Self-evolving reasoning LLM from zero data,” arXiv:2508.05004, 2025.
- [17] G. Wang *et al.*, “Voyager: An open-ended embodied agent with large language models,” arXiv:2305.16291, 2023.
- [18] N. Shinn *et al.*, “Reflexion: Language agents with verbal reinforcement learning,” arXiv:2303.11366, 2023.